Python Programming



PROF. DR. SOUMEN BANERJEE

Ph.D (IIEST-S), M.Tech (CU), B.Tech (CU), B.Sc (Hons-CU)
Senior Member - IEEE, Fellow-IETE, Life Member InRaSS

Faculty – School of Computing and Information Technology
EASTERN INTERNATIONAL UNIVERSITY (EIU) - VIETNAM
81, Nam Ky Khoi Nghia Street, Binh Duong Ward, HCMC, Vietnam.

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

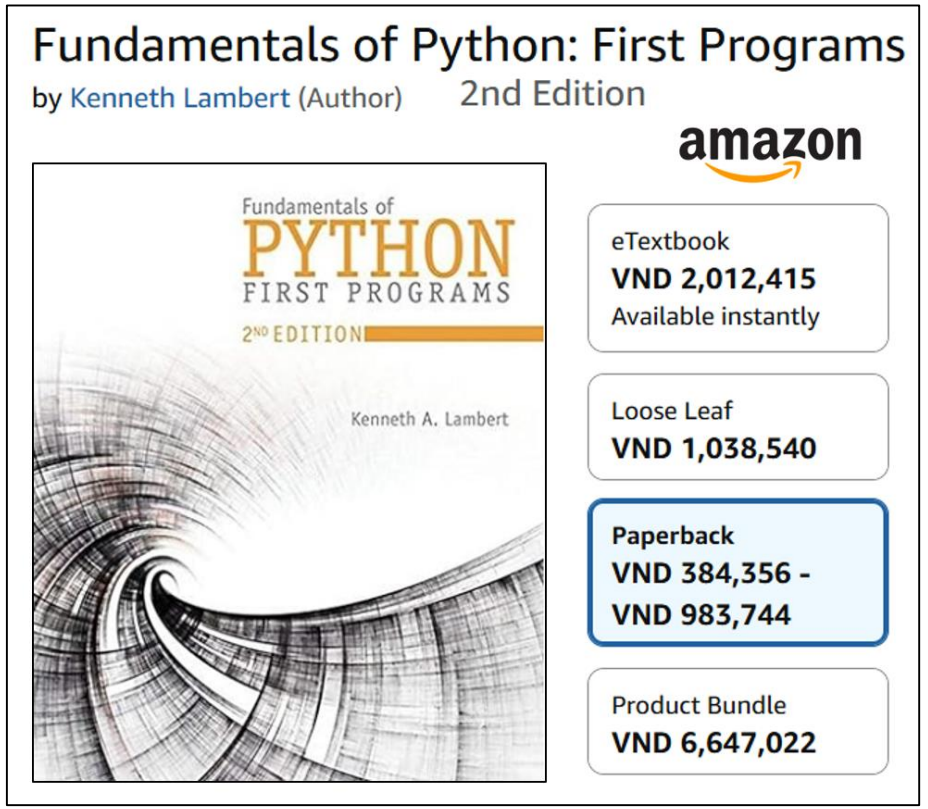
python™



PYTHON PROGRAMMING

EIU
EASTERN INTERNATIONAL UNIVERSITY

Fundamentals of Python Programming





EIU
EASTERN
INTERNATIONAL
UNIVERSITY





EIU
EASTERN INTERNATIONAL UNIVERSITY

[APPENDIX] A

Python Resources

Table A.1 provides information on an excellent Web site where programmers can find complete documentation for the Python API (Application Programming Interface) and download Python and other resources.

DESCRIPTION	URL	EXPLANATION
Python's top-level Web page	*http://www.python.org/*	This page contains news about events in the Python world and links to documentation, Python-related products, program examples, and free downloads of resources.
Downloads	*http://www.python.org/download/*	This page allows you to select the version of Python that matches your computer and to begin the download process.
Documentation and training	*http://www.python.org/doc/*	This page allows you to browse the documentation for the Python API, tutorials, and other training aids. You can also download many of these items to your computer for offline reference.

[TABLE A.1] Online Python Documentation



A.1

Installing Python on Your Computer

As of this writing, Python does not come preinstalled on Windows systems. Therefore, you must download the Windows installer from *http://www.python.org/download/*. The installer might then run automatically, or you might have to double-click an icon for the installer to launch it. The installer automatically puts Python into a folder and inserts various command options on the **All Programs** menu. Note that administrators installing Python for all users on Windows Vista need to be logged in as Administrator.

Macintosh users running Mac OS X might need to update the version of Python that comes preinstalled on their systems. A Mac OS X installer can be downloaded for this purpose and behaves in a manner similar to that of the Windows installer.

Unix and Linux users also might need to upgrade the version of Python that comes preinstalled on their systems. In these cases, they have to download a compressed Python source code "tarball" from the same site and install it.

Most users will also want to place aliases of the important Python commands, such as the one that launches IDLE, on their desktops.

[CHAPTER] 1 | Introduction

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

Guido van Rossum *invented the* ***Python*** *programming language in the early 1990s. Python is a high-level, general purpose programming language for solving problems on modern computer systems. The language and many supporting tools are free, and Python programs can run on any operating system.*

You can download Python, its documentation, and related materials from www.python.org.

Guido van Rossum



Van Rossum in 2024

Born	31 January 1956 (age 70)[1] The Hague, Netherlands[2]
Alma mater	University of Amsterdam (BS)
Occupations	Computer programmer, author
Employer	Microsoft
Known for	Creating the Python programming language
Children	1[3]
Awards	Award for the Advancement of Free Software (2001)
Website	gvanrossum.github.io

EIU
EASTERN INTERNATIONAL UNIVERSITY

Running Code in the Interactive Shell

Python is an interpreted language, and you can run simple Python expressions and statements in an interactive programming environment called the **shell**. The easiest way to open a Python shell is to launch the **IDLE (Integrated DeveLopment Environment).** This is an integrated program development environment that comes with the Python installation. When you do this, a window named Python Shell opens. Figure 1-6 shows a shell window on macOS. A shell window running on a Windows system or a Linux system should look similar, if not identical, to this one.

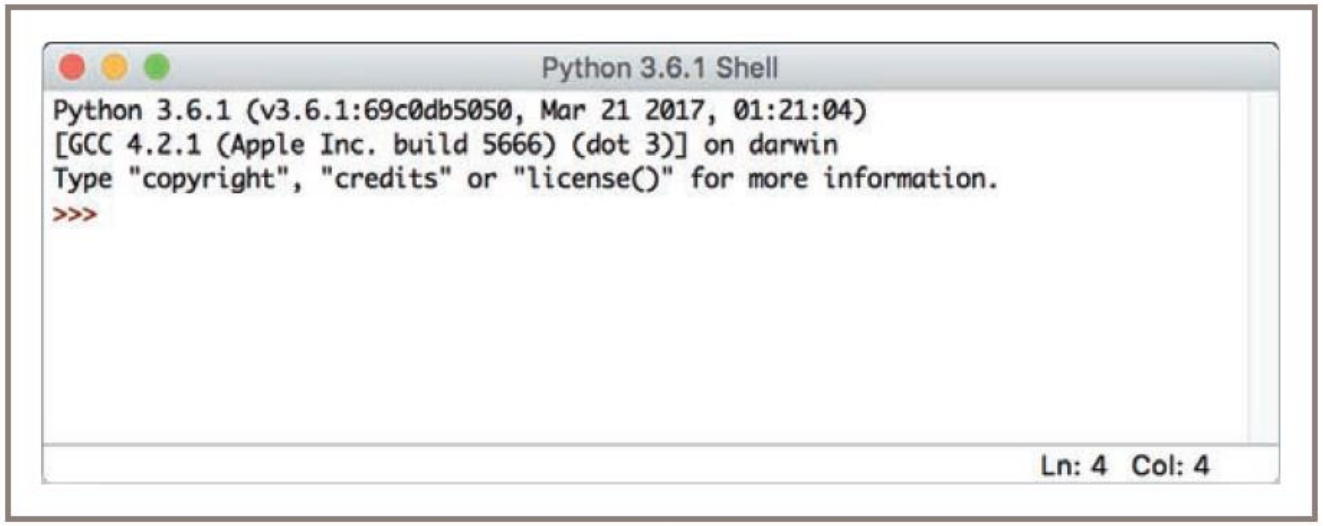


Figure 1-6 Python shell window

EIU
EASTERN INTERNATIONAL UNIVERSITY

A shell window contains an opening message followed by the special symbol >>>, called a shell prompt. The cursor at the shell prompt waits for you to enter a Python command. Note that you can get immediate **help** by entering help at the shell prompt or selecting **Help** from the window's drop-down menu.

```
>>> 3 + 4                          # Simple arithmetic
7
>>> 3                              # The value of 3 is
3
>>> "Python is really cool!"       # Use a string for text
'Python is really cool!'
>>> name = "Ken Lambert"           # Give a variable a value
>>> name                           # The value of name is
'Ken Lambert'
>>> "Hi there, " + name            # Create some new text
'Hi there, Ken Lambert'
>>> print('Hi there')              # Output some text
Hi there
>>> print("Hi there,", name)       # Output two values
Hi there, Ken Lambert
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Color	Type of Element	Examples
Black	Inputs in the IDLE shell Numbers Operator symbols Variable, function, and method references Punctuation marks	`67, +, name, y = factorial(x)`
Blue	Outputs in the IDLE shell Function, class, and method names in definitions	`'Ken Lambert'`, `def factorial(n)`
Green	Strings	`"Ken Lambert"`
Orange	Keywords	`def, if, while`
Purple	Built-in function names	`abs, round, int`
Red	Program comments Error messages in the IDLE shell	`# Output the results` `ZeroDivisionError: division by zero`

EIU
EASTERN INTERNATIONAL UNIVERSITY

Input, Processing, and Output

The programmer can also force the output of a value by using the **print** function. The simplest form for using this function looks like the following:

```
print(<expression>)
```

This example shows you the basic **syntax** (or grammatical rule) for using the **print** function. The angle brackets (the < and > symbols) enclose a type of phrase. In actual Python code, you would replace this syntactic form, including the angle brackets, with an example of that type of phrase.

In this example, the text **"Hi there"** is the text that we want Python to display. In programming terminology, this piece of text is referred to as a **string**. In Python code, a string is always enclosed in quotation marks. However, the **print** function displays a string without the quotation marks.

```
>>> print ("Hi there")
Hi there
```

You can also write a **print** function that includes two or more expressions separated by commas. In such a case, the **print** function evaluates the expressions and displays their results, separated by single spaces, on one line. The syntax for a **print** statement with two or more expressions looks like the following:

```
print(<expression>,..., <expression>)
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
print(<expression>, end = "")
```

Whether it outputs one or multiple expressions, the **print** function always ends its output with a **newline**. In other words, it displays the values of the expressions, and then it moves the cursor to the next line on the console window.

To begin the next output on the same line as the previous one, you can place the expression **end = ""**, which says "end the line with an empty string instead of a newline," at the end of the list of expressions, as follows:

```
>>> name = input("Enter your name: ")
Enter your name: Ken Lambert
>>> name
'Ken Lambert'
>>> print(name)
Ken Lambert
>>>
```

The following example receives an input string from the user and saves it for further processing. The user's input is in black.

The **input** function does the following:

1. Displays a prompt for the input. In this example, the prompt is **"Enter your name: "**.
2. Receives a string of keystrokes, called characters, entered at the keyboard and returns the string to the shell.

EIU
EASTERN INTERNATIONAL UNIVERSITY

The text in parentheses, **"Enter your name: "**, is an argument for the **input** function that tells it what to use for the prompt. An **argument** is a piece of information that a function needs to do its work.

The string returned by the function in our example is saved by assigning it to the variable **name**. The form of an assignment statement with the **input** function is the following:

```
<variable identifier> = input(<a string prompt>)
```

A **variable identifier**, or **variable** for short, is just a name for a value. When a variable receives its value in an input statement, the variable then refers to this value. If the user enters the name **"Ken Lambert"** in our last example, the value of the variable **name** can be viewed as follows:

```
>>> name
'Ken Lambert'
```

The **input** function always builds a string from the user's keystrokes and returns it to the program. After inputting strings that represent numbers, the programmer must convert them from strings to the appropriate numeric types. In Python, there are two **type conversion functions** for this purpose, called **int** (for integers) and **float** (for floating-point numbers).

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> first = int(input("Enter the first number: "))
Enter the first number: 23
>>> second = int(input("Enter the second number: "))
Enter the second number: 44
>>> print("The sum is", first + second)
The sum is 67
```

Note that the **int** function is called with each result returned by the **input** function. The two numbers are added, and then their sum is output.

Table 1-2 Basic Python functions for input and output

Function	What It Does
`float(<a string of digits>)`	Converts a string of digits to a floating-point value.
`int(<a string of digits>)`	Converts a string of digits to an integer value.
`input(<a string prompt>)`	Displays the string prompt and waits for keyboard input. Returns the string of characters entered by the user.
`print(<expression>, ...,<expression>)`	Evaluates the expressions and displays them, separated by one space, in the console window.
`<string 1> + <string 2>`	Glues the two strings together and returns the result.

Editing, Saving, and Running a Script

To compose and execute programs in this manner, you perform the following steps:

1. Select the option **New Window** from the **File** menu of the shell window.
2. In the new window, enter Python expressions or statements on separate lines, in the order in which you want Python to execute them.
3. At any point, you may save the file by selecting File/Save. If you do this, you should use a **.py** extension. For example, your first program file might be named **myprogram.py**.
4. To run this file of code as a Python script, select **Run Module** from the **Run** menu or press the F5 key.

myprogram.py.

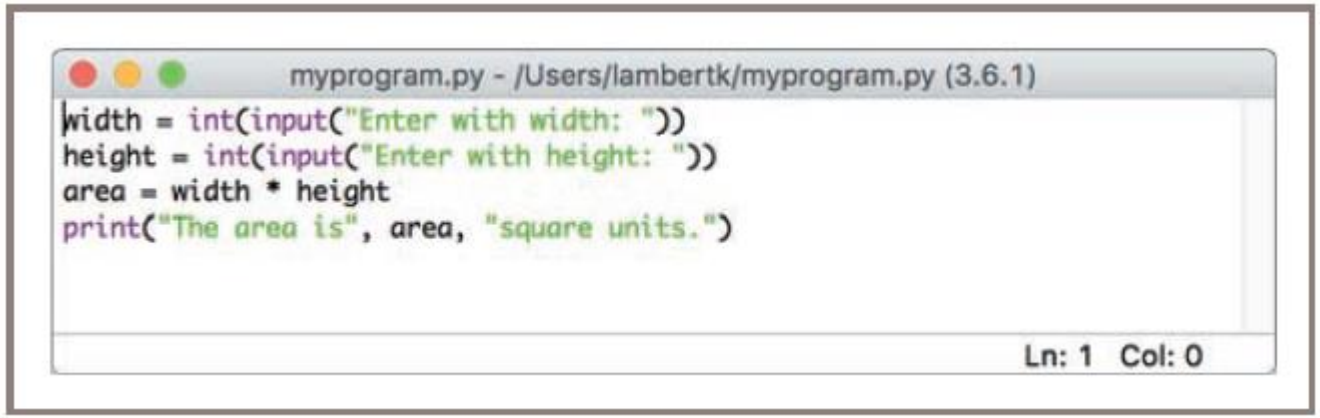


Figure 1-7 Python script in an IDLE window

EIU EASTERN INTERNATIONAL UNIVERSITY

Behind the Scenes: How Python Works

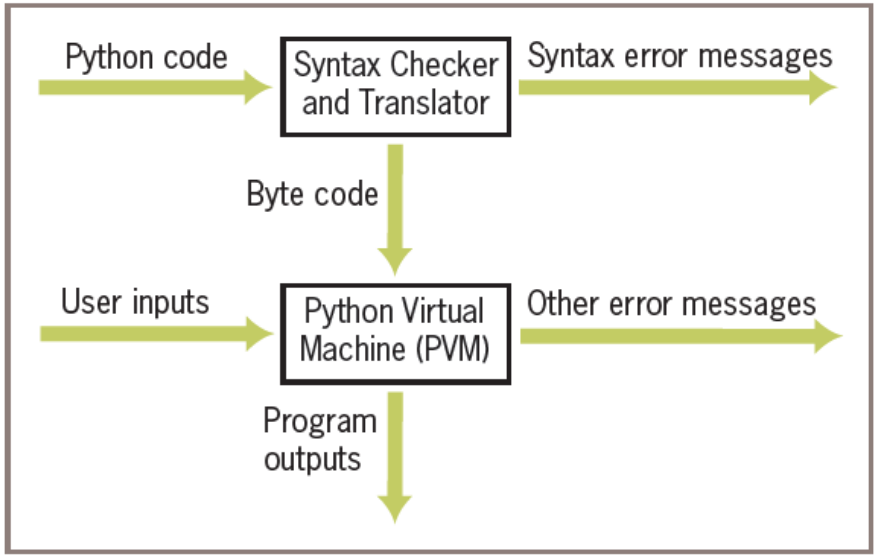


Figure 1-9 Steps in interpreting a Python program

1. The interpreter reads a Python expression or statement, also called the **source code**, and verifies that it is well formed. In this step, the interpreter behaves like a strict English teacher who rejects any sentence that does not adhere to the grammar rules, or syntax, of the language. As soon as the interpreter encounters such an error, it halts translation with an error message.
2. If a Python expression is well formed, the interpreter then translates it to an equivalent form in a low-level language called **byte code**. When the interpreter runs a script, it completely translates it to byte code.
3. This byte code is next sent to another software component, called the **Python virtual machine (PVM)**, where it is executed. If another error occurs during this step, execution also halts with an error message.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Detecting and Correcting Syntax Errors

The term *syntax* refers to the rules for forming sentences in a language. When Python encounters a syntax error in a program, it halts execution with an error message.

```
>>> length = int(input("Enter the length: "))
Enter the length: 44
>>> print(lenth)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
NameError: name 'lenth' is not defined
```

The first statement assigns an input value to the variable **length**. The next statement attempts to print the value of the variable **lenth**. Python responds that this name is not defined. Although the programmer might have *meant* to write the variable **length**, Python can read only what the programmer *actually entered*.'

The next statement attempts to print the value of the correctly spelled variable. However, Python still generates an error message.

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> print(length)
SyntaxError: unexpected indent
```

In this error message, Python explains that this line of code is unexpectedly indented. In fact, there is an extra space before the word **print**. Indentation is significant in Python code. Each line of code entered at a shell prompt or in a script must begin in the leftmost column, with no leading spaces.

In our final example, the programmer attempts to add two numbers, but forgets to include the second one:

```
>>> 3 +
SyntaxError: invalid syntax
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Open a Python shell, enter the following expressions, and observe the results:

   a. **8**
   b. **8 * 2**
   c. **8 ** 2**
   d. **8/12**
   e. **8 // 12**
   f. **8/0**

2. Write a Python program that prints (displays) your name, address, and telephone number.

3. Evaluate the following code at a shell prompt: `print ("Your name is", name)`. Then assign `name` an appropriate value, and evaluate the statement again.

4. Open an IDLE window, and enter the program from Figure 1-7 that computes the area of a rectangle. Load the program into the shell by pressing the F5 key, and correct any errors that occur. Test the program with different inputs by running it at least three times.

5. Modify the program of Project 4 to compute the area of a triangle. Issue the appropriate prompts for the triangle's base and height, and change the names of the variables appropriately. Then, use the formula `.5 * base * height` to compute the area. Test the program from an IDLE window.

EIU
EASTERN INTERNATIONAL UNIVERSITY



Exercises

6. Write and test a program that computes the area of a circle. This program should request a number representing a radius as input from the user. It should use the formula `3.14 * radius ** 2` to compute the area and then output this result suitably labeled.

7. Write and test a program that accepts the user's name (as text) and age (as a number) as input. The program should output a sentence containing the user's name and age.

8. Enter an input statement using the `input` function at the shell prompt. When the prompt asks you for input, enter a number. Then, attempt to add 1 to that number, observe the results, and explain what happened.

9. Enter an input statement using the `input` function at the shell prompt. When the prompt asks you for input, enter your first name, observe the results, and explain what happened.

EIU
EASTERN INTERNATIONAL UNIVERSITY

[CHAPTER] 2

SOFTWARE DEVELOPMENT,

Data Types, and Expressions

EIU
EASTERN
INTERNATIONAL
UNIVERSITY

Data Types

In programming, a **data type** consists of a set of values and a set of operations that can be performed on those values. A **literal** is the way a value of a data type looks to a programmer.

Table 2-2 Literals for some Python data types

Type of Data	Python Type Name	Example Literals
Integers	`int`	`-1, 0, 1, 2`
Real numbers	`float`	`-0.55, .3333, 3.14, 6.0`
Character strings	`str`	`"Hi", "", 'A', "66"`

The first two data types listed in Table 2-2, `int` and `float`, are called **numeric data types**, because they represent numbers.

EIU
EASTERN INTERNATIONAL UNIVERSITY

String Literals

In Python, a string literal is a sequence of characters enclosed in single or double quotation marks.

The last two string literals ('' and "") represent the **empty string**. Although it contains no characters, the empty string is a string nonetheless. Note that the empty string is different from a string that contains a single blank space character, " ".

Double-quoted strings are handy for composing strings that contain single quotation marks or apostrophes. Here is a self-justifying example:

```
>>> 'Hello there!'
'Hello there!'
>>> "Hello there!"
'Hello there!'
>>> ''
''
>>> ""
''
```

```
>>> "I'm using a single quote in this string!"
"I'm using a single quote in this string!"
>>> print("I'm using a single quote in this string!")
I'm using a single quote in this string!
```

Note that the **`print`** function displays the nested quotation mark but not the enclosing quotation marks. A double quotation mark can also be included in a string literal if one uses the single quotation marks to enclose the literal.

EIU
EASTERN INTERNATIONAL UNIVERSITY

```
>>> print("""This very long sentence extends
all the way to the next line.""")
This very long sentence extends
all the way to the next line.
```

Note that the first line in the output ends exactly where the first line ends in the code.

When you evaluate a string in the Python shell without the **print** function, you can see the literal for the **newline character**, **\n**, embedded in the result, as follows:

```
>>> """This very long sentence extends
all the way to the next line."""
'This very long sentence extends\nall the way to the next line.'
```

Escape Sequences

The newline character **\n** is called an **escape sequence**. Escape sequences are the way Python expresses special characters, such as the tab, the newline, and the backspace (delete key), as literals. Table 2-3 lists some escape sequences in Python.

Table 2-3

Some escape sequences in Python

Escape Sequence	Meaning
\b	Backspace
\n	Newline
\t	Horizontal tab
\\	The \ character
\'	Single quotation mark
\"	Double quotation mark

EIU
EASTERN INTERNATIONAL UNIVERSITY

Because the backslash is used for escape sequences, it must be escaped to appear as a literal character in a string. Thus, **print('\\')** would display a single \ character.

String Concatenation

You can join two or more strings to form a new string using the concatenation operator +. Here is an example:

```
>>> "Hi " + "there, " + "Ken!"
'Hi there, Ken!'
```

The * operator allows you to build a string by repeating another string a given number of times. The left operand is a string, and the right operand is an integer. For example, if you want the string **"Python"** to be preceded by 10 spaces, it would be easier to use the * operator with 10 and one space than to enter the 10 spaces by hand.

```
>>> " " * 10 + "Python"
'          Python'
```

Variables and the Assignment Statement

Variables receive their initial values and can be reset to new values with an **assignment statement**. The simplest form of an assignment statement is the following:

```
<variable name> = <expression>
```

```
>>> firstName = "Ken"
>>> secondName = "Lambert"
>>> fullName = firstName + " " + secondName
>>> fullName
'Ken Lambert'
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Program Comments and Docstrings

We conclude this subsection on strings with a discussion of **program comments**. A comment is a piece of program text that the computer ignores but that provides useful documentation to programmers. At the very least, the author of a program can include his or her name and a brief statement about the program's purpose at the beginning of the program file. This type of comment, called a **docstring**, is a multi-line string of the form discussed earlier in this section. Here is a docstring that begins a typical program for a lab session:

```
"""
Program: circle.py
Author: Ken Lambert
Last date modified: 10/10/17

The purpose of this program is to compute the area of a
circle. The input is an integer or floating-point number
representing the radius of the circle. The output is a
floating-point number labeled as the area of the circle.
"""
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

In addition to docstrings, **end-of-line comments** can document a program. These comments begin with the # symbol and extend to the end of a line. An end-of-line comment might explain the purpose of a variable or the strategy used by a piece of code, if it is not already obvious. Here is an example:

```
>>> RATE = 0.85 # Conversion rate for Canadian to US dollars
```

Throughout this book, docstrings appear in green and end-of-line comments appear in red.

Exercises



1. Let the variable **x** be **"dog"** and the variable **y** be **"cat"**. Write the values returned by the following operations:

   a. `x + y`

   b. `"the " + x + " chases the " + y`

   c. `x * 4`

2. Write a string that contains your name and address on separate lines using embedded newline characters. Then write the same string literal without the newline characters.

3. How does one include an apostrophe as a character within a string literal?

4. What happens when the **print** function prints a string literal with embedded newline characters?

5. Which of the following are valid variable names?

   a. `length` b. `_width` c. `firstBase` d. `2MoreToGo` e. `halt!`

EIU
EASTERN INTERNATIONAL UNIVERSITY

Numeric Data Types and Character Sets

Integers

As you learned in mathematics, the **integers** include 0, the positive whole numbers, and the negative whole numbers. Integer literals in a Python program are written without commas, and a leading negative sign indicates a negative value.

Although the range of integers is infinite, a real computer's memory places a limit on the magnitude of the largest positive and negative integers. The most common implementation of the `int` data type in many programming languages consists of the integers from −2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$).

Floating-Point Numbers

A real number in mathematics, such as the value of π (3.1416...), consists of a whole number, a decimal point, and a fractional part. Real numbers have **infinite precision**, which means that the digits in the fractional part can continue forever. Like the integers, real numbers also have an infinite range.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Python uses **floating-point** numbers to represent real numbers. Values of the most common implementation of Python's `float` type range from approximately -10^{308} to 10^{308} and have 16 digits of precision.

A floating-point number can be written using either ordinary **decimal notation** or **scientific notation**. Scientific notation is often useful for mentioning very large numbers. Table 2-4 shows some equivalent values in both notations.

Table 2-4 Decimal and scientific notations for floating-point numbers

Decimal Notation	Scientific Notation	Meaning
3.78	3.78e0	3.78×10^{0}
37.8	3.78e1	3.78×10^{1}
3780.0	3.78e3	3.78×10^{3}
0.378	3.78e–1	3.78×10^{-1}
0.00378	3.78e–3	3.78×10^{-3}

EIU
EASTERN INTERNATIONAL UNIVERSITY

Character Sets

Some programming languages use different data types for strings and individual characters. In Python, character literals look just like string literals and are of the string type. To mark the difference in this book, we use single quotes to enclose single-character strings, and double quotes to enclose multi-character strings. Thus, we refer to `'H'` as a character and `"Hi!"` as a string, even though they are both technically Python strings, and both are color-coded in green in this text.

all data and instructions in a program are translated to binary numbers before being run on a real computer. To support this translation, the characters in a string each map to an integer value. This mapping is defined in character sets, among them the **ASCII set** and the **Unicode set**. (The term ASCII stands for American Standard Code for Information Interchange.)

EIU
EASTERN INTERNATIONAL UNIVERSITY

ASCII set (The term ASCII stands for American Standard Code for Information Interchange.) In the 1960s, the original ASCII set encoded each keyboard character and several control characters using the integers from 0 through 127. An example of a control character is Control+D, which is the command to terminate a shell window. As new function keys and some international characters were added to keyboards, the ASCII set doubled in size to 256 distinct values in the mid-1980s.
Table 2-5 shows the mapping of character values to the first 128 ASCII codes.

Table 2-5 The original ASCII character set

	0	1	2	3	4	5	6	7	8	9
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT
1	LF	VT	FF	CR	SO	SI	DLE	DCI	DC2	DC3
2	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS
3	RS	US	SP	!	"	#	$	%	&	`
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	I	j	k	I	m
11	n	o	P	q	r	S	t	u	v	w
12	X	y	z	{	|	}	~	DEL		

Python's **ord** and **chr** functions convert characters to their numeric ASCII codes and back again, respectively.

```
>>> ord('a')
97
>>> ord('A')
65
>>> chr(65)
'A'
>>> chr(66)
'B'
```

Exercises



1. Which data type would most appropriately be used to represent the following data values?
   a. The number of months in a year
   b. The area of a circle
   c. The current minimum wage
   d. The approximate age of the universe (12,000,000,000 years)
   e. Your name
2. Explain the differences between the data types **`int`** and **`float`**.
3. Write the values of the following floating-point numbers in Python's scientific notation:
   a. 355.76
   b. 0.007832
   c. 4.3212

EIU
EASTERN INTERNATIONAL UNIVERSITY

Expressions

Arithmetic Expressions

Table 2-6 Arithmetic operators

Operator	Meaning	Syntax
-	Negation	-a
**	Exponentiation	a ** b
*	Multiplication	a * b
/	Division	a / b
//	Quotient	a // b
%	Remainder or modulus	a % b
+	Addition	a + b
-	Subtraction	a - b

Expression	Evaluation	Value
5 + 3 * 2	5 + 6	11
(5 + 3) * 2	8 * 2	16
6 % 2	0	0
2 * 3 ** 2	2 * 9	18
-3 ** 2	-(3 ** 2)	-9
(3) ** 2	9	9
2 ** 3 ** 2	2 ** 9	512
(2 ** 3) ** 2	8 ** 2	64
45 / 0	Error: cannot divide by 0	
45 % 0	Error: cannot divide by 0	

The last two lines of Table 2-7 show attempts to divide by 0, which result in an error.

A **semantic error** is detected when the action that an expression describes cannot be carried out, even though that expression is syntactically correct. Although the expressions **45 / 0** and **45 % 0** are syntactically correct, they are meaningless, because a computing agent cannot carry them out.

The quotient operator **//** produces an integer quotient, whereas the exact division operator **/** always produces a **float**. Thus, **3 // 4** produces **0**, whereas **3 / 4** produces **.75**.

```
>>> 3 + 4 * \
2 ** 5
131
```

Although spacing within an expression is not important to the Python interpreter, programmers usually insert a single space before and after each operator to make the code easier for people to read. Normally, an expression must be completed on a single line of Python code. When an expression becomes long or complex, you can move to a new line by placing a backslash character \ at the end of the current line. The next example shows this technique:

Arithmetic Expressions

```
>>> 3.14 * 3 ** 2
28.26
```

```
>>> radius = input("Enter the radius: ")
Enter the radius: 3.2
>>> radius
'3.2'
>>> float(radius)
3.2
>>> float(radius) ** 2 * 3.14
32.153600000000004
```

Note that the **int** function converts a **float** to an **int** by truncation, not by rounding to the nearest whole number. Truncation simply chops off the number's fractional part. The **round** function rounds a **float** to the nearest **int** as in the next example:

```
>>> int(6.75)
6
>>> round(6.75)
7
```

Conversion Function	Example Use	Value Returned
int(<a number or a string>)	int(3.77)	3
	int("33")	33
float(<a number or a string>)	float(22)	22.0
str(<any value>)	str(99)	'99'

EIU
EASTERN INTERNATIONAL UNIVERSITY

Python is a **strongly typed programming language**.

Python does not allow the use of the + operator with a string and a number:

```
>>> profit = 1000.55
>>> print('$' + profit)

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'float' objects
```

To solve this problem, we use the **str** function to convert the value of **profit** to a string and then concatenate this string to the **$** symbol, as follows:

```
>>> print('$' + str(profit))
$1000.55
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



1. Let `x` = 8 and `y` = 2. Write the values of the following expressions:

   a. `x + y * 3`

   b. `(x + y) * 3`

   c. `x ** y`

   d. `x % y`

   e. `x / 12.0`

   f. `x // 6`

2. Let `x` = 4.66 Write the values of the following expressions:

   a. `round(x)`

   b. `int(x)`

3. How does a Python programmer round a `float` value to the nearest `int` value?

4. How does a Python programmer concatenate a numeric value to a string value?

5. Assume that the variable `x` has the value 55. Use an assignment statement to increment the value of `x` by 1.

EIU
EASTERN INTERNATIONAL UNIVERSITY

Using Functions and Modules

We can apply an operator such as + to one or more operands to produce a new data value. Alternatively, we can call a function such as **round** with one or more data values to produce a new data value. Python includes many useful functions, which are organized in libraries of code called **modules**.

Calling Functions: Arguments and Return Values

A **function** is a chunk of code that can be called by name to perform a task. Functions often require **arguments**, that is, specific data values, to perform their tasks. Names that refer to arguments are also known as **parameters**. When a function completes its task (which is usually some kind of computation), the function may send a result back to the part of the program that called that function in the first place. The process of sending a result back to another part of a program is known as **returning a value**.

For example, the argument in the function call **round(6.5)** is the value **6.5**, and the value returned is **7**. When an argument is an expression, it is first evaluated, and then its value is passed to the function for further processing. For instance, the function call **abs(4 - 5)** first evaluates the expression **4 - 5** and then passes the result, **-1**, to **abs**. Finally, **abs** returns **1**.

The values returned by function calls can be used in expressions and statements. For example, the function call **print(abs(4 - 5) + 3)** prints the value **4**.

Some functions have only **optional arguments**, some have **required arguments**, and some have both required and optional arguments. For example, the **round** function has one required argument, the number to be rounded. When called with just one argument, the **round** function exhibits its **default behavior**, which is to return the nearest whole number with a fractional part of 0. However, when a second, optional argument is supplied, this argument, a number, indicates the number of places of precision to which the first argument should be rounded. For example, **round(7.563, 2)** returns **7.56**.

```
>>> help(round)
Help on built-in function round in module builtin:

round(...)
    round(number[, ndigits]) -> floating point number
```

The math Module

Functions and other resources are coded in components called **modules**. Functions like **abs** and **round** from the **__builtin__** module are always available for use, whereas the programmer must explicitly import other functions from the modules where they are defined.

The **math** module includes several functions that perform basic mathematical operations. The next code session imports the **math** module and lists a directory of its resources:

```
>>> import math
>>> dir(math)
['__doc__', '__file__', '__loader__', '__name__',
'__package__', '__spec__', 'acos', 'acosh', 'asin',
'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign',
'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp',
'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp',
'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose',
'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log',
'log10', 'log1p', 'log2', 'modf', 'nan', 'pi', 'pow',
'radians', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau',
'trunc']
```

EIU
EASTERN INTERNATIONAL UNIVERSITY

This list of function names includes some familiar trigonometric functions as well as Python's most exact estimates of the constants π and **e**.

To use a resource from a module, you write the name of a module as a qualifier, followed by a dot (.) and the name of the resource. For example, to use the value of **pi** from the **math** module, you would write the following code: **math.pi**. The next session uses this technique to display the value of π and the square root of 2:

```
>>> math.pi
3.1415926535897931
>>> math.sqrt(2)
1.4142135623730951
```

Once again, help is available if needed:

```
>>> help(math.cos)
Help on built-in function cos in module math:
cos(...)
    cos(x)

    Return the cosine of x (measured in radians).
```

help(math). The function **help** uses a module's own docstring and the docstrings of all its functions to print the documentation.

EIU
EASTERN INTERNATIONAL UNIVERSITY

For example, the statement **from math import *** would import all of the **math** module's resources.

```
>>> from math import pi, sqrt
>>> print(pi, sqrt(2))
3.14159265359 1.41421356237
```

The Main Module

In the case study, earlier in this chapter, we showed how to write documentation for a Python script. To differentiate this script from the other modules in a program (and there could be many), we call it the **main module**. Like any module, the main module can also be imported. Instead of launching the script from a terminal prompt or loading it into the shell from IDLE, you can start IDLE from the terminal prompt and import the script as a module.

```
>>> import taxform
Enter the gross income: 120000
Enter the number of dependents: 2
The income tax is $20800.0
```


Exercises

1. The **math** module includes a **pow** function that raises a number to a given power. The first argument is the number, and the second argument is the exponent. Write a code segment that imports this function and calls it to print the values 8^2 and 5^4.



EIU
EASTERN INTERNATIONAL UNIVERSITY

Exercises



2. You can calculate the surface area of a cube if you know the length of an edge. Write a program that takes the length of an edge (an integer) as input and prints the cube's surface area as output.

3. Write a program that takes the radius of a sphere (a floating-point number) as input and then outputs the sphere's diameter, circumference, surface area, and volume.

4. An object's momentum is its mass multiplied by its velocity. Write a program that accepts an object's mass (in kilograms) and velocity (in meters per second) as inputs and then outputs its momentum.

5. Write a program that calculates and prints the number of minutes in a year.

6. Light travels at $3 * 10^8$ meters per second. A light-year is the distance a light beam travels in one year. Write a program that calculates and displays the value of a light-year.

EIU
EASTERN INTERNATIONAL UNIVERSITY